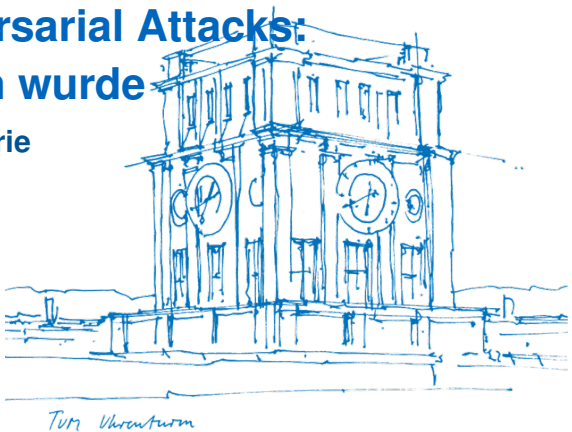
Neuronale Netze und Adversarial Attacks: Wie der Panda zum Gibbon wurde

Data Science in Forschung und Industrie

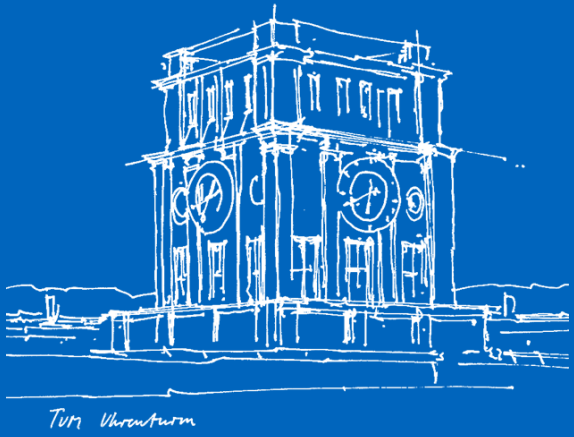
Tim Roith

M15
TUM School of Computation, Information and Technology
Technische Universität München

29. Oktober 2026



Klassifizierung mit neuronalen Netzen



Bildklassifizierung: Was sind Bilder?

- Wir konzentrieren uns in diesem Vortrag auf Bildklassifizierung.

Bildklassifizierung: Was sind Bilder?

- Wir konzentrieren uns in diesem Vortrag auf Bildklassifizierung.
- Viele der folgenden Konzepte lassen sich auf andere Aufgaben übertragen.

Bildklassifizierung: Was sind Bilder?

- Wir konzentrieren uns in diesem Vortrag auf Bildklassifizierung.
- Viele der folgenden Konzepte lassen sich auf andere Aufgaben übertragen.
- Wir repräsentieren ein Farbbild als ein Element

$$x \in [0,1]^{3\times H\times W}$$

mit $H, W \in \mathbb{N}$ (drei Farbkanäle, Höhe H, Breite W).

Bildklassifizierung: Was sind Bilder?

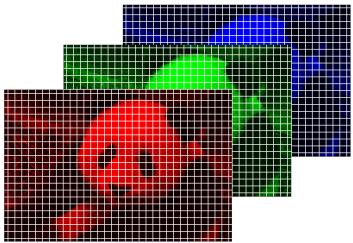
- Wir konzentrieren uns in diesem Vortrag auf Bildklassifizierung.
- Viele der folgenden Konzepte lassen sich auf andere Aufgaben übertragen.
- Wir repräsentieren ein Farbbild als ein Element

$$x \in [0,1]^{3\times H\times W}$$

mit $H, W \in \mathbb{N}$ (drei Farbkanäle, Höhe H, Breite W).







Bildklassifizierung: Was ist Klassifizierung?

- Wir betrachten eine Menge möglicher Klassen

$$\mathcal{C} = \{\text{Panda, Hydrant, Spaghetti, } \ldots, \text{PKW}\}.$$

Bildklassifizierung: Was ist Klassifizierung?

- Wir betrachten eine Menge möglicher Klassen

$$\mathcal{C} = \{\text{Panda, Hydrant, Spaghetti, } \ldots, \text{PKW}\}.$$

- Wir möchten nun eine Abbildung f finden, die jedem Bild x eine Klasse zuordnet:

$$f\left(\text{  }\right) = \text{Panda}$$

Bildklassifizierung: Was ist Klassifizierung?

- Wir betrachten eine Menge möglicher Klassen

$$\mathcal{C} = \{\text{Panda, Hydrant, Spaghetti, } \ldots, \text{ PKW}\}.$$

- Wir möchten nun eine Abbildung f finden, die jedem Bild x eine Klasse zuordnet:

$f\left(\right.$  $\left.\right) = \text{Panda}$

- Oftmals gibt f nicht direkt eine Klasse zurück, sondern einen Vektor mit einem Wert pro Klasse

$$f(x) = (0.6, \ldots, 0.3, \ldots, 0.1) \in \mathbb{R}^{|\mathcal{C}|}.$$

Panda Gibbon PKW

Bildklassifizierung: Wie gut ist der Klassifizierer?

- Wir befinden uns im „supervised“ Setting, d. h. uns ist ein Datensatz von Input-Output-Paaren gegeben,

$$\mathcal{T} = \{(x_1, \text{PKW}), (x_2, \text{Katze}), \ldots, (x_N, \text{Panda})\}.$$

Bildklassifizierung: Wie gut ist der Klassifizierer?

- Wir befinden uns im „supervised“ Setting, d. h. uns ist ein Datensatz von Input-Output-Paaren gegeben,

$$\mathcal{T} = \{(x_1, \text{PKW}), (x_2, \text{Katze}), \ldots, (x_N, \text{Panda})\}.$$

- Wir können die Güte des Klassifizierers f nun messen: Für ein Paar $(x, y) \in \mathcal{T}$ betrachten wir den Fehler

$$\ell(f(x), y).$$

  Typische Wahl für $\ell(\cdot, \cdot)$ (mit y als Vektor kodiert, z. B. Panda $\widehat{=}\, (1, 0, \ldots, 0)$): $\ell(y', y) = \|y' - y\|$ oder $\ell(\cdot, \cdot) =$ Kreuzentropie.

Bildklassifizierung: Wie gut ist der Klassifizierer?

- Wir befinden uns im „supervised“ Setting, d. h. uns ist ein Datensatz von Input-Output-Paaren gegeben,

$$\mathcal{T} = \{(x_1, \text{PKW}), (x_2, \text{Katze}), \dots, (x_N, \text{Panda})\}.$$

- Wir können die Güte des Klassifizierers f nun messen: Für ein Paar $(x, y) \in \mathcal{T}$ betrachten wir den Fehler

$$\ell(f(x), y).$$

  Typische Wahl für $\ell(\cdot, \cdot)$ (mit y als Vektor kodiert, z. B. Panda $\widehat{=}\, (1, 0, \dots, 0)$): $\ell(y', y) = \|y' - y\|$ oder $\ell(\cdot, \cdot) =$ Kreuzentropie.

- Der Gesamtfehler auf allen Daten ist dann gegeben durch

$$\sum_{(x,y)\in\mathcal{T}} \ell(f(x), y).$$

Bildklassifizierung: Wie lernt man den Klassifizierer?

- Um einen Klassifizierer verändern zu können, betrachten wir Parameter $\theta \in \Theta$ und eine Parametrisierung f_θ.

Bildklassifizierung: Wie lernt man den Klassifizierer?

- Um einen Klassifizierer verändern zu können, betrachten wir Parameter $\theta \in \Theta$ und eine Parametrisierung f_θ.
- Beispiel lineare Funktion: $f_\theta(x) = Wx + b$ mit Parametern $\theta = (W, b)$.

Bildklassifizierung: Wie lernt man den Klassifizierer?

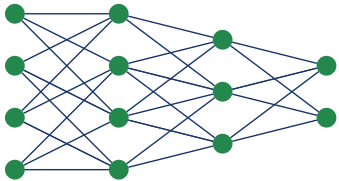
- Um einen Klassifizierer verändern zu können, betrachten wir Parameter $\theta \in \Theta$ und eine Parametrisierung f_θ.
- Beispiel lineare Funktion: $f_\theta(x) = Wx + b$ mit Parametern $\theta = (W, b)$.
- Die Funktion zu lernen bedeutet, Parameter θ zu finden, sodass die **Loss**-Funktion

$$\mathcal{L}(\theta) = \sum_{(x,y)\in\mathcal{T}} \ell(f_\theta(x), y)$$

minimiert wird.

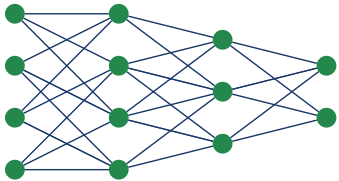
Parametrisierung durch neuronale Netze

- Wir parametrisieren den Klassifizierer im Folgenden als **neuronales Netz**.



Parametrisierung durch neuronale Netze

- Wir parametrisieren den Klassifizierer im Folgenden als **neuronales Netz**.



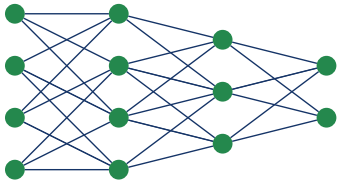
- Ein neuronales Netz besteht aus Schichten (Layern) der Form

$$\Phi_i(z) := \sigma(W_i z + b_i),$$

W_i: Gewichtsmatrix, b_i: Bias, σ: Aktivierungsfunktion.

Parametrisierung durch neuronale Netze

- Wir parametrisieren den Klassifizierer im Folgenden als **neuronales Netz**.



- Ein neuronales Netz besteht aus Schichten (Layern) der Form

$$\Phi_i(z) := \sigma(W_i z + b_i),$$

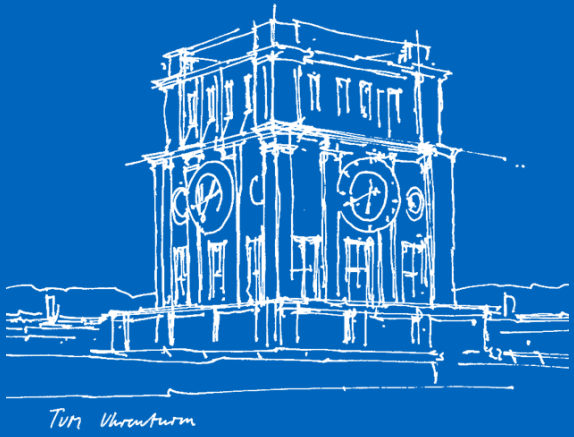
  W_i: Gewichtsmatrix, b_i: Bias, σ: Aktivierungsfunktion.

- Das gesamte Netz entsteht durch Hintereinanderausführung aller Schichten

$$f_\theta = \Phi_L \circ \cdots \circ \Phi_2 \circ \Phi_1$$

  mit Parametern $\theta = ((W_L, b_L), \ldots, (W_2, b_2), (W_1, b_1))$.

Adversarial Attacks



Wie gut funktionieren neuronale Netze?

- Positive Resultate (um 2015):

Wie gut funktionieren neuronale Netze?

- Positive Resultate (um 2015):
  - He, Zhang, Ren und Sun: *„Delving deep into rectifiers: Surpassing human-level performance on imagenet classification“*

Wie gut funktionieren neuronale Netze?

- Positive Resultate (um 2015):
  - He, Zhang, Ren und Sun: *„Delving deep into rectifiers: Surpassing human-level performance on imagenet classification“*
  - He, Zhang, Ren und Sun: *„Deep residual learning for image recognition“*
    (siehe auch Srivastava, Greff und Schmidhuber: *„Highway networks“*)

Wie gut funktionieren neuronale Netze?

- Positive Resultate (um 2015):
  - He, Zhang, Ren und Sun: *„Delving deep into rectifiers: Surpassing human-level performance on imagenet classification“*
  - He, Zhang, Ren und Sun: *„Deep residual learning for image recognition“*
    (siehe auch Srivastava, Greff und Schmidhuber: *„Highway networks“*)
- Bedenkliche Resultate (schon etwas früher, 2013/14):

Wie gut funktionieren neuronale Netze?

- Positive Resultate (um 2015):
  - He, Zhang, Ren und Sun: *„Delving deep into rectifiers: Surpassing human-level performance on imagenet classification“*
  - He, Zhang, Ren und Sun: *„Deep residual learning for image recognition“*
    (siehe auch Srivastava, Greff und Schmidhuber: *„Highway networks“*)
- Bedenkliche Resultate (schon etwas früher, 2013/14):
  - Szegedy u. a.: *„Intriguing properties of neural networks“*

Wie gut funktionieren neuronale Netze?

- Positive Resultate (um 2015):
  - He, Zhang, Ren und Sun: *„Delving deep into rectifiers: Surpassing human-level performance on imagenet classification“*
  - He, Zhang, Ren und Sun: *„Deep residual learning for image recognition“*
    (siehe auch Srivastava, Greff und Schmidhuber: *„Highway networks“*)
- Bedenkliche Resultate (schon etwas früher, 2013/14):
  - Szegedy u. a.: *„Intriguing properties of neural networks“*
  - I. J. Goodfellow, Shlens und Szegedy: *„Explaining and harnessing adversarial examples“*

Was sind Adversarial Attacks?

Das berühmteste Beispiel aus [GSS14]:

Was sind Adversarial Attacks?

Das berühmteste Beispiel aus [GSS14]:



$\boldsymbol{x}$

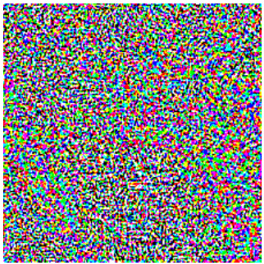
“panda”
57.7% confidence

Was sind Adversarial Attacks?

Das berühmteste Beispiel aus [GSS14]:



$+ .007 \times$



$\boldsymbol{x}$

"panda"
57.7% confidence

$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"nematode"
8.2% confidence

Was sind Adversarial Attacks?

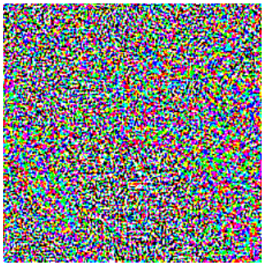
Das berühmteste Beispiel aus [GSS14]:



$\boldsymbol{x}$

"panda"
57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"nematode"
8.2% confidence

$=$



$\boldsymbol{x} + \epsilon\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"gibbon"
99.3 % confidence

Was sind Adversarial Attacks?

- Gegeben sei ein Bild x, das von f_θ „korrekt“ klassifiziert wird [Bun+23].

Was sind Adversarial Attacks?

- Gegeben sei ein Bild x, das von f_θ „korrekt“ klassifiziert wird [Bun+23].

- Ein (untargeted) Adversarial Example ist ein Bild $\tilde{x}$ „nahe“ bei x, sodass f_θ das Bild $\tilde{x}$ falsch klassifiziert.

Was sind Adversarial Attacks?

- Gegeben sei ein Bild x, das von f_θ „korrekt“ klassifiziert wird [Bun+23].
- Ein (untargeted) Adversarial Example ist ein Bild $\tilde{x}$ „nahe“ bei x, sodass f_θ das Bild $\tilde{x}$ falsch klassifiziert.
- Intuitiv bedeutet „nahe“:

Was sind Adversarial Attacks?

- Gegeben sei ein Bild x, das von f_θ „korrekt“ klassifiziert wird [Bun+23].
- Ein (untargeted) Adversarial Example ist ein Bild $\tilde{x}$ „nahe“ bei x, sodass f_θ das Bild $\tilde{x}$ falsch klassifiziert.
- Intuitiv bedeutet „nahe“:
  - $\tilde{x}$ sieht ähnlich aus wie x, der Unterschied ist *kaum wahrnehmbar*,

Was sind Adversarial Attacks?

- Gegeben sei ein Bild x, das von f_θ „korrekt“ klassifiziert wird [Bun+23].
- Ein (untargeted) Adversarial Example ist ein Bild $\tilde{x}$ „nahe“ bei x, sodass f_θ das Bild $\tilde{x}$ falsch klassifiziert.
- Intuitiv bedeutet „nahe“:
  - $\tilde{x}$ sieht ähnlich aus wie x, der Unterschied ist *kaum wahrnehmbar*,
  - ein Mensch würde x und $\tilde{x}$ gleich klassifizieren.

Was sind Adversarial Attacks?

- Gegeben sei ein Bild x, das von f_θ „korrekt“ klassifiziert wird [Bun+23].

- Ein (untargeted) Adversarial Example ist ein Bild $\tilde{x}$ „nahe“ bei x, sodass f_θ das Bild $\tilde{x}$ falsch klassifiziert.

- Intuitiv bedeutet „nahe“:
  - $\tilde{x}$ sieht ähnlich aus wie x, der Unterschied ist *kaum wahrnehmbar*,
  - ein Mensch würde x und $\tilde{x}$ gleich klassifizieren.

- In der Praxis wählen wir oft eine ℓ^p-Norm, um den Abstand zu messen,

$$\|\tilde{x} - x\|_p^p = \sum_i |\tilde{x}_i - x_i|^p \,, \qquad \|\tilde{x} - x\|_\infty = \max_i |\tilde{x}_i - x_i| \,,$$

  wobei i alle Einträge (Kanal, Zeile, Spalte) des Bildes durchläuft.

Sind Adversarial Attacks rein additiv?

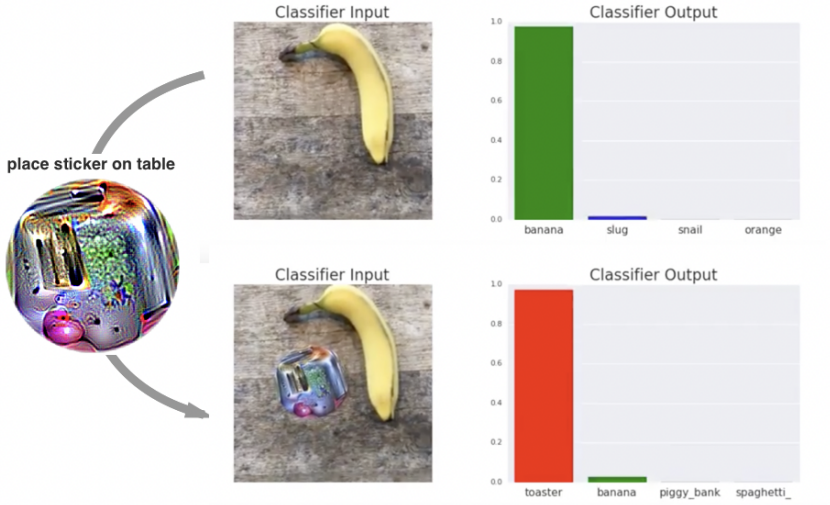


Clean Image
35.3% Soccer Ball
7.4% Pick
4.3% Golf Ball



Adversarial Example
46.1% Shower Cap
9.2% Bonnet
8.0% Tissue

Adversarial Patches: [Bro+17]



Wie erzeugt man Adversarial Examples?

Gegeben seien ein Bild x mit wahrem Label y und ein *Budget* $\varepsilon > 0$. Wir betrachten das Problem

$$\max_{\tilde{x}:\|\tilde{x}-x\|\leq\varepsilon} \ell(f_\theta(\tilde{x}), y).$$

Wie erzeugt man Adversarial Examples?

Gegeben seien ein Bild x mit wahrem Label y und ein *Budget* $\varepsilon > 0$. Wir betrachten das Problem

$$\max_{\tilde{x}:\|\tilde{x}-x\|\leq\varepsilon} \ell(f_\theta(\tilde{x}), y).$$

Projizierter Gradientenaufstieg (PGD) [KGB+16; Mad+17], ausgehend von $\tilde{x}^0 = x$:

$$\tilde{x}^{k+1} = \Pi_{B_\varepsilon(x)} \left(\tilde{x}^k + \eta \nabla_{\tilde{x}} \ell(f_\theta(\tilde{x}^k), y) \right).$$

Ein einziger Schritt (mit Vorzeichen des Gradienten) ist die „Fast Gradient Sign Method“ aus [GSS14].

Wie erzeugt man Adversarial Examples?

Gegeben seien ein Bild x mit wahrem Label y und ein *Budget* $\varepsilon > 0$. Wir betrachten das Problem

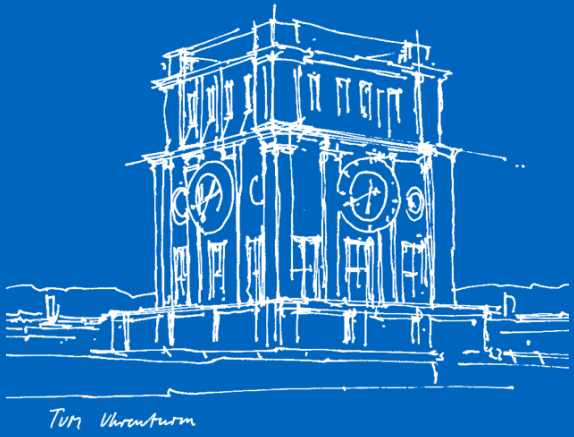
$$\max_{\tilde{x}:\|\tilde{x}-x\|\leq\varepsilon} \ell(f_\theta(\tilde{x}), y).$$

Projizierter Gradientenaufstieg (PGD) [KGB+16; Mad+17], ausgehend von $\tilde{x}^0 = x$:

$$\tilde{x}^{k+1} = \Pi_{B_\varepsilon(x)} \left(\tilde{x}^k + \eta \nabla_{\tilde{x}} \ell(f_\theta(\tilde{x}^k), y) \right).$$

Ein einziger Schritt (mit Vorzeichen des Gradienten) ist die „Fast Gradient Sign Method“ aus [GSS14].
„Open-Box“ (White-Box): Wir können die Gradienten des Netzes auswerten.
„Closed-Box“ (Black-Box): Gradienten des Netzes werden nicht benutzt [Ily+18].

Verteidigung gegen Adversarial Attacks



Adversarial Training
[GSS14; Mad+17]

Folgende Arbeiten:

- I. J. Goodfellow, Shlens und Szegedy: *„Explaining and harnessing adversarial examples“*
- Madry, Makelov, Schmidt, Tsipras und Vladu: *„Towards deep learning models resistant to adversarial attacks“*

Adversarial Training
[GSS14; Mad+17]

Folgende Arbeiten:

- I. J. Goodfellow, Shlens und Szegedy: *„Explaining and harnessing adversarial examples“*
- Madry, Makelov, Schmidt, Tsipras und Vladu: *„Towards deep learning models resistant to adversarial attacks“*

schlagen „Adversarial Training“ vor, d. h. das Netz wird direkt auf Adversarial Examples trainiert:

$$\min_{\theta} \mathbb{E}_{(x,y)} \max_{\delta \in B_\varepsilon(0)} \ell(f_\theta(x+\delta), y).$$

Eine andere Idee: Lipschitz-Stetigkeit

Lipschitz-Stetigkeit: Es gibt ein $L \geq 0$, sodass

$$(\forall x, \tilde{x} \in \mathcal{X}) \quad \|f_\theta(x) - f_\theta(\tilde{x})\|_{\mathcal{Y}} \leq L \|x - \tilde{x}\|_{\mathcal{X}} .$$

Eine andere Idee: Lipschitz-Stetigkeit

Lipschitz-Stetigkeit: Es gibt ein $L \geq 0$, sodass

$$(\forall x, \tilde{x} \in \mathcal{X}) \quad \|f_\theta(x) - f_\theta(\tilde{x})\|_{\mathcal{Y}} \leq L \|x - \tilde{x}\|_{\mathcal{X}} .$$

Die (kleinste) Lipschitz-Konstante ist dann gegeben durch

$$\mathrm{Lip}(f_\theta) := \sup_{x, \tilde{x} \in \mathcal{X},\ x \neq \tilde{x}} \frac{\|f_\theta(x) - f_\theta(\tilde{x})\|_{\mathcal{Y}}}{\|x - \tilde{x}\|_{\mathcal{X}}}.$$

Eine andere Idee: Lipschitz-Stetigkeit

Lipschitz-Stetigkeit: Es gibt ein $L \geq 0$, sodass

$$(\forall x, \tilde{x} \in \mathcal{X}) \quad \|f_\theta(x) - f_\theta(\tilde{x})\|_{\mathcal{Y}} \leq L \, \|x - \tilde{x}\|_{\mathcal{X}} \, .$$

Die (kleinste) Lipschitz-Konstante ist dann gegeben durch

$$\mathrm{Lip}(f_\theta) := \sup_{x, \tilde{x} \in \mathcal{X},\ x \neq \tilde{x}} \frac{\|f_\theta(x) - f_\theta(\tilde{x})\|_{\mathcal{Y}}}{\|x - \tilde{x}\|_{\mathcal{X}}}.$$

Warum hilft das? Aus $\|\tilde{x} - x\|_{\mathcal{X}} \leq \varepsilon$ folgt $\|f_\theta(\tilde{x}) - f_\theta(x)\|_{\mathcal{Y}} \leq L \, \varepsilon$,
d. h. kleine Störungen der Eingabe ändern die Ausgabe nur wenig.

Regularisierung mit der Lipschitz-Konstante

Wir betrachten

$$\min_{\theta \in \Theta} \frac{1}{|\mathcal{T}|} \sum_{(x,y) \in \mathcal{T}} \ell(f_\theta(x), y) + \lambda \operatorname{Lip}(f_\theta).$$

Regularisierung mit der Lipschitz-Konstante

Wir betrachten

$$\min_{\theta \in \Theta} \frac{1}{|\mathcal{T}|} \sum_{(x,y) \in \mathcal{T}} \ell(f_\theta(x), y) + \lambda \operatorname{Lip}(f_\theta).$$

Aber: Die exakte Berechnung von $\operatorname{Lip}(f_\theta)$ ist NP-schwer [SV18].

Regularisierung mit der Lipschitz-Konstante

Wir betrachten

$$\min_{\theta \in \Theta} \frac{1}{|\mathcal{T}|} \sum_{(x,y) \in \mathcal{T}} \ell(f_\theta(x), y) + \lambda \operatorname{Lip}(f_\theta).$$

Aber: Die exakte Berechnung von $\operatorname{Lip}(f_\theta)$ ist NP-schwer [SV18].

Alternativ: Benutze die Abschätzung

$$\operatorname{Lip}(f_\theta) \;\leq\; \prod_{l=1}^{L} \operatorname{Lip}(\Phi_l) \leq C_\sigma \prod_{l=1}^{L} \|W_l\| \,.$$

Regularisierung mit der Lipschitz-Konstante

Wir betrachten

$$\min_{\theta \in \Theta} \frac{1}{|\mathcal{T}|} \sum_{(x,y) \in \mathcal{T}} \ell(f_\theta(x), y) + \lambda \operatorname{Lip}(f_\theta).$$

Aber: Die exakte Berechnung von $\operatorname{Lip}(f_\theta)$ ist NP-schwer [SV18].

Alternativ: Benutze die Abschätzung

$$\operatorname{Lip}(f_\theta) \;\leq\; \prod_{l=1}^{L} \operatorname{Lip}(\Phi_l) \leq C_\sigma \prod_{l=1}^{L} \|W_l\| \,.$$

Aber: Diese Abschätzung überschätzt $\operatorname{Lip}(f_\theta)$ oft deutlich [ALG19; Gou+20; KMP20; RKH19].

Lipschitz-Training: CLIP [Bun+21]

Die Idee aus [Bun+21]: Approximiere die Lipschitz-Konstante $\mathrm{Lip}(f_\theta)$ auf einer **endlichen, aber variablen** Menge $\mathcal{X}_{\mathrm{Lip}} \subset \mathcal{X} \times \mathcal{X}$:

$$\mathrm{Lip}(f_\theta) \approx \max_{(x,\tilde{x}) \in \mathcal{X}_{\mathrm{Lip}}} \frac{\|f_\theta(x) - f_\theta(\tilde{x})\|}{\|x - \tilde{x}\|}.$$

Adversarial Updates: Toy Example

Adversarial Updates: Toy Example

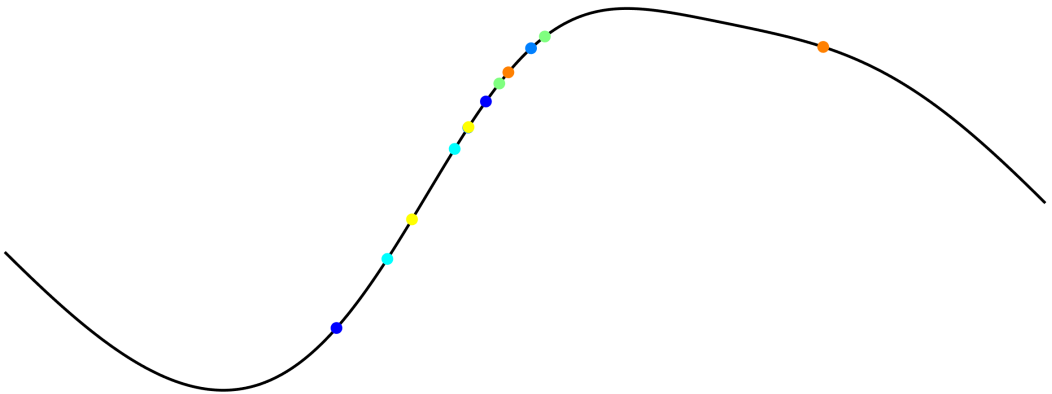


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

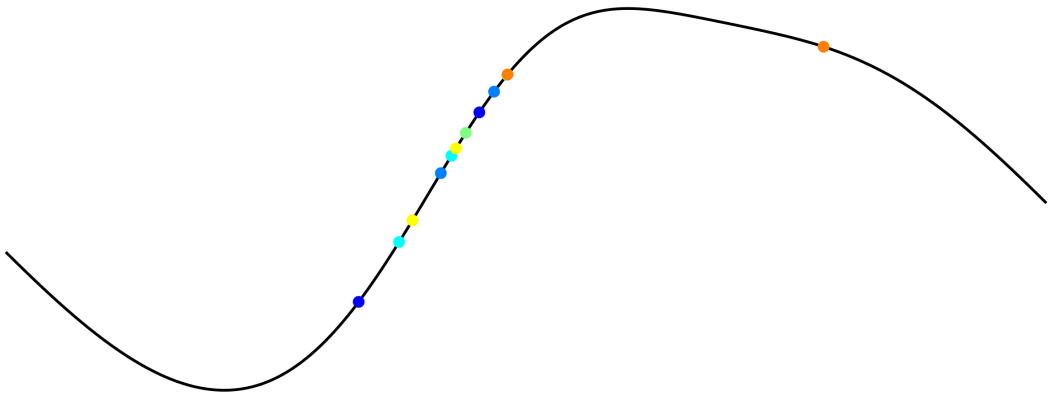


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

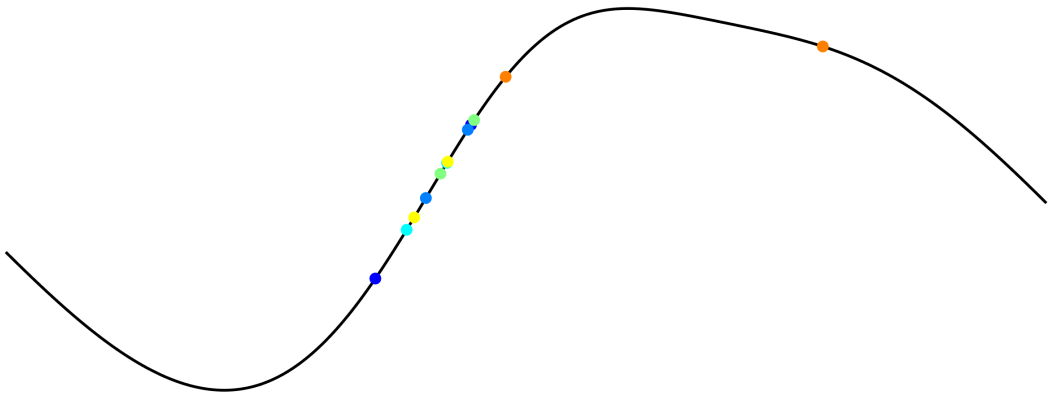


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

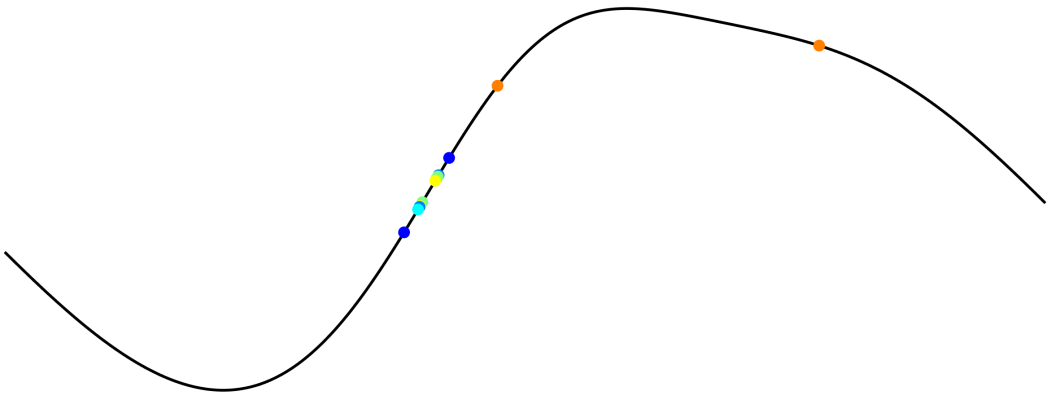


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

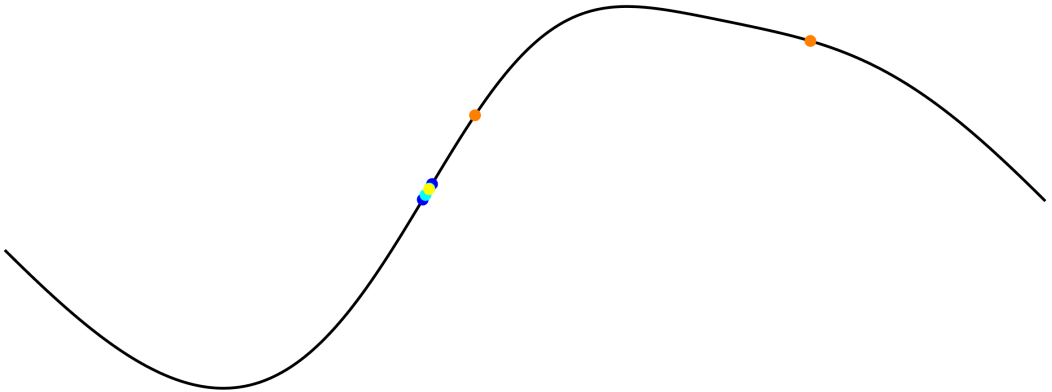


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

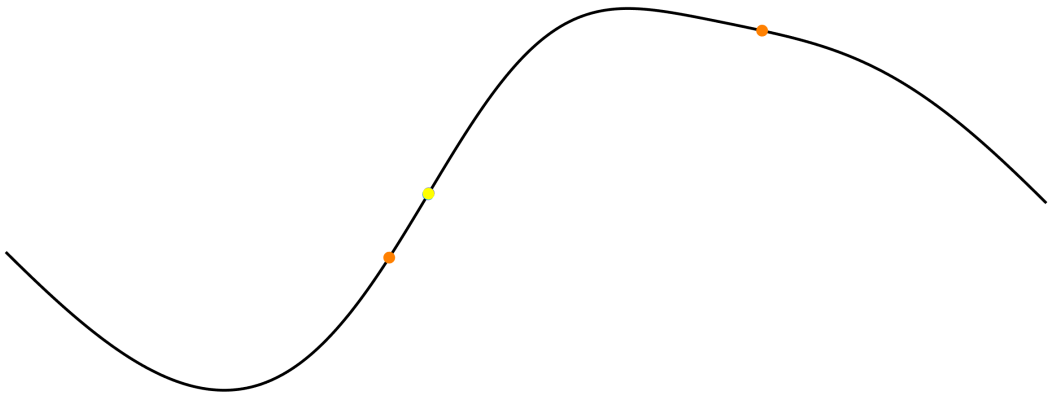


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

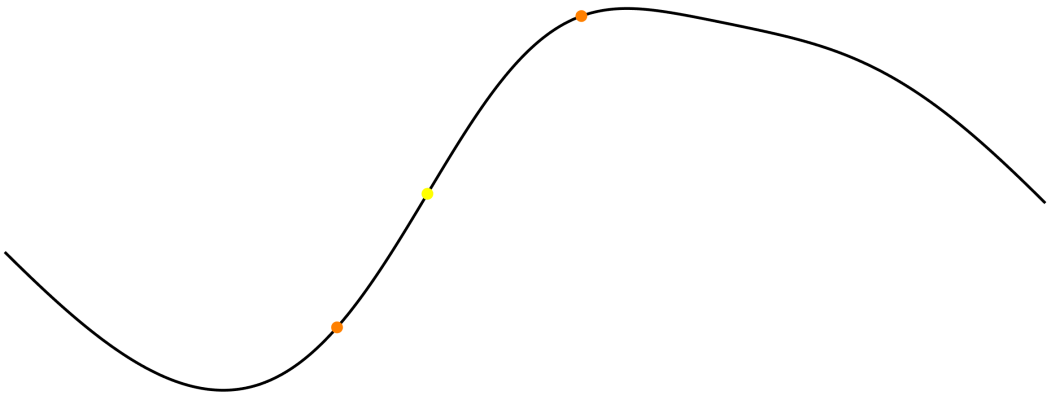


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

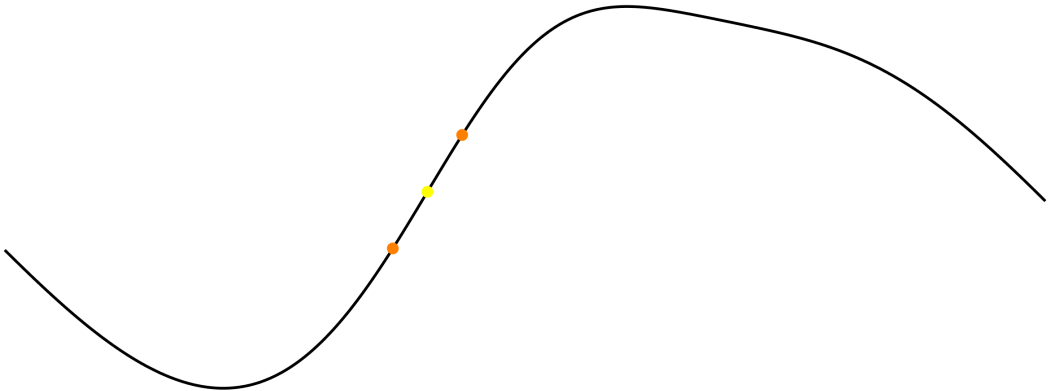


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

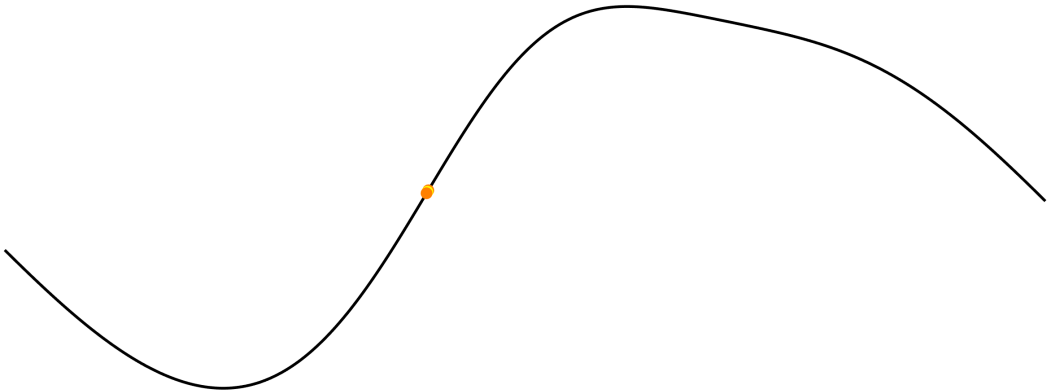


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Toy Example

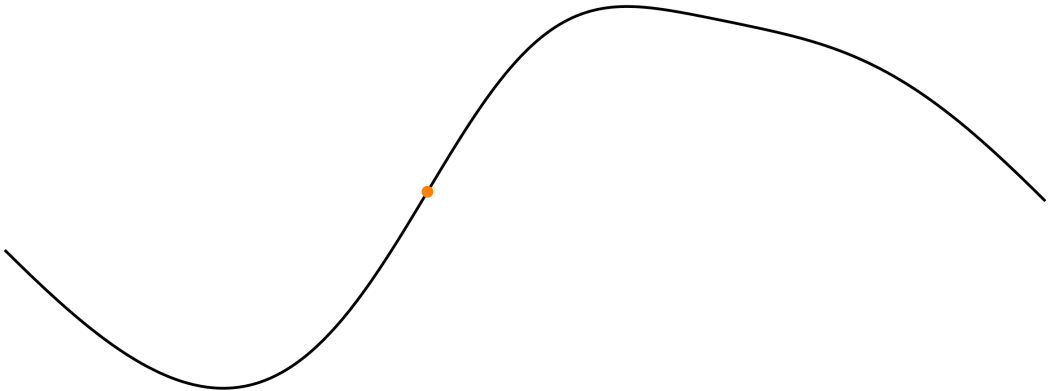
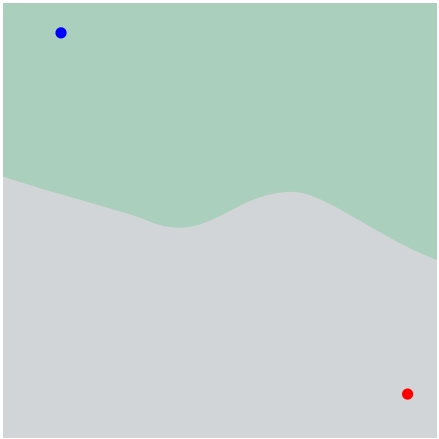
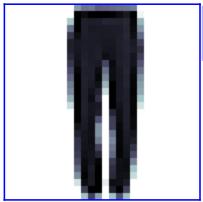


Abbildung 1 Adversariale Paare für eine 1D-Funktion.

Adversarial Updates: Fashion-MNIST





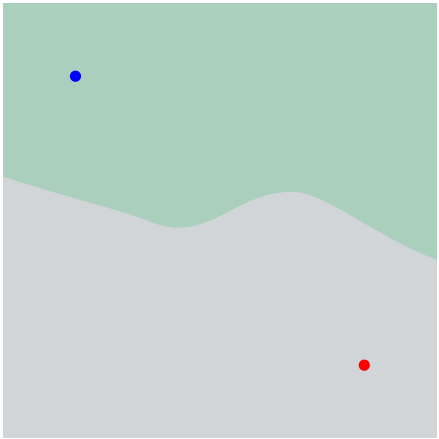
Trouser
Confidence: 100.0%

Lip constant: 0.01



Shirt
Confidence: 100.0%

Adversarial Updates: Fashion-MNIST





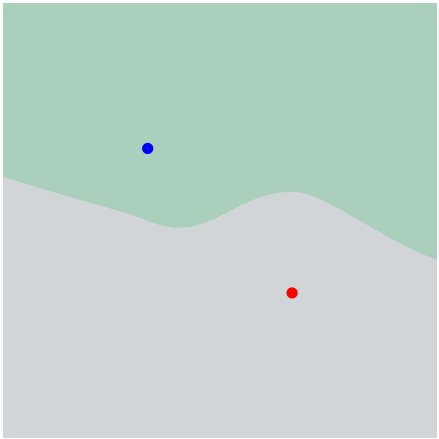
Trouser
Confidence: 100.0%

Lip constant: 0.01



Shirt
Confidence: 100.0%

Adversarial Updates: Fashion-MNIST





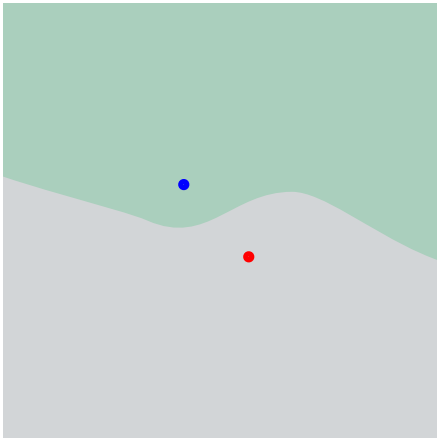
Trouser
Confidence: 100.0%

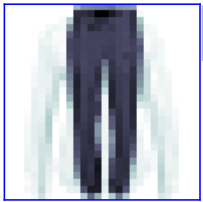
Lip constant: 0.02



Shirt
Confidence: 100.0%

Adversarial Updates: Fashion-MNIST





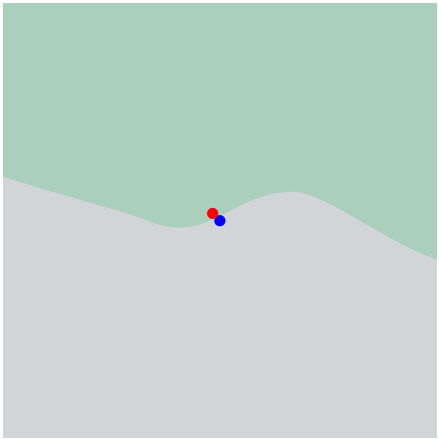
Trouser
Confidence: 100.0%

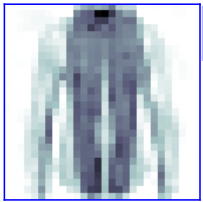
Lip constant: 0.04



Shirt
Confidence: 68.52%

Adversarial Updates: Fashion-MNIST





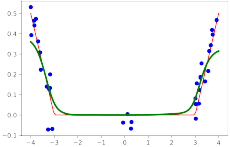
Shirt
Confidence: 99.92%

Lip constant: 17.21



Trouser
Confidence: 99.26%

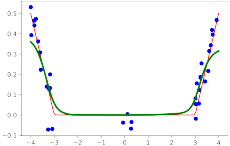
Regression in 1D



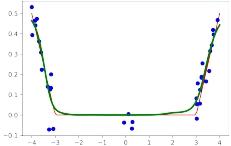
(a) $\lambda = 10$

Rot: wahre Funktion, **Blau**: verrauschte Daten, **Grün**: trainierte Netze für verschiedene Regularisierungsparameter λ.

Regression in 1D



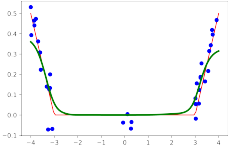
(a) $\lambda = 10$



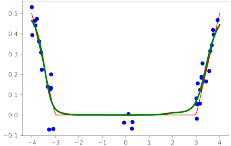
(b) $\lambda = 1$

Rot: wahre Funktion, **Blau**: verrauschte Daten, **Grün**: trainierte Netze für verschiedene Regularisierungsparameter λ.

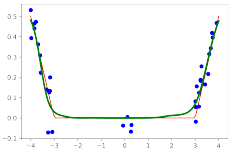
Regression in 1D



(a) $\lambda = 10$



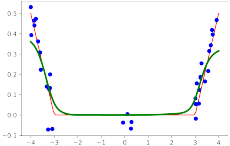
(b) $\lambda = 1$



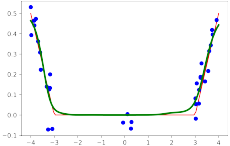
(c) $\lambda = 10^{-10}$

Rot: wahre Funktion, **Blau**: verrauschte Daten, **Grün**: trainierte Netze für verschiedene Regularisierungsparameter λ.

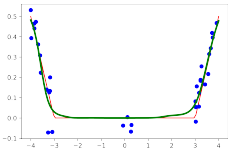
Regression in 1D



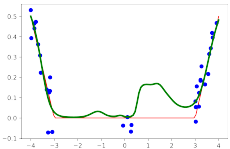
(a) $\lambda = 10$



(b) $\lambda = 1$



(c) $\lambda = 10^{-10}$



(d) $\lambda = 0$

Rot: wahre Funktion, **Blau**: verrauschte Daten, **Grün**: trainierte Netze für verschiedene Regularisierungsparameter λ.

Einige empirische Ergebnisse

	Training Type	Train	Test	Noise	PGD	μ, λ
MNIST	Standard	**95.6**	89.8	71.8	25.3	0
	Weight Reg.$_{95}$	93.1	89.0	71.4	25.4	0.0003
	Weight Reg.$_{90}$	89.8	89.7	71.9	24.8	0.0015
	Weight Reg.$_{85}$	84.8	87.2	72.3	24.9	0.0031
	CLIP$_{95}$	95.3	90.7	70.3	24.4	0.01
	CLIP$_{90}$	91.8	**91.6**	**78.0**	36.2	3.91
	CLIP$_{85}$	87.6	87.2	74.5	**37.7**	9.82
Fashion-MNIST	Standard	**98.5**	**92.0**	25.7	1.9	0
	Weight Reg.$_{95}$	94.8	91.2	26.9	3.5	0.0011
	Weight Reg.$_{90}$	89.8	89.6	28.1	6.0	0.0023
	Weight Reg.$_{85}$	85.3	85.4	**34.2**	10.2	0.0091
	CLIP$_{95}$	94.6	**91.9**	19.8	9.1	0.18
	CLIP$_{90}$	90.6	88.9	23.3	13.0	0.74
	CLIP$_{85}$	85.9	83.1	31.9	**14.9**	4.18

Vielen Dank!

Fragen?

Dieser Vortrag basiert auf: L. Bungert, R. Raab, T. R, L. Schwinn und D. Tenbrinck. „CLIP: Cheap Lipschitz Training of Neural Networks“. In: *Scale Space and Variational Methods in Computer Vision*. 2021

Vielen Dank!
Fragen?

Dieser Vortrag basiert auf: L. Bungert, R. Raab, T. R, L. Schwinn und D. Tenbrinck. „CLIP: Cheap Lipschitz Training of Neural Networks“. In: *Scale Space and Variational Methods in Computer Vision*. 2021

Der Code ist auf GitHub verfügbar:
`https://github.com/TimRoith/CLIP`



Literatur I

[ALG19] C. Anil, J. Lucas und R. B. Grosse. „Sorting Out Lipschitz Function Approximation“. In: *ICML*. Bd. 97. PMLR. 2019, S. 291–301.

[Bro+17] T. B. Brown, D. Mané, A. Roy, M. Abadi und J. Gilmer. „Adversarial patch“. In: *arXiv preprint arXiv:1712.09665* (2017).

[Bun+21] L. Bungert, R. Raab, T. R, L. Schwinn und D. Tenbrinck. „CLIP: Cheap Lipschitz Training of Neural Networks“. In: *Scale Space and Variational Methods in Computer Vision*. 2021.

[Bun+23] L. Bungert, N. G. Trillos, M. Jacobs, D. McKenzie, . Nikoli und Q. Wang. „It begins with a boundary: A geometric view on probabilistically robust learning“. In: *arXiv preprint arXiv:2305.18779* (2023).

Literatur II

[Gou+20] H. Gouk, E. Frank, B. Pfahringer und M. J. Cree. „Regularisation of neural networks by enforcing Lipschitz continuity“. In: *Machine Learning* (2020), S. 1–24.

[GSS14] I. J. Goodfellow, J. Shlens und C. Szegedy. „Explaining and harnessing adversarial examples“. In: *arXiv preprint arXiv:1412.6572* (2014).

[He+15] K. He, X. Zhang, S. Ren und J. Sun. „Delving deep into rectifiers: Surpassing human-level performance on imagenet classification“. In: *Proceedings of the IEEE international conference on computer vision*. 2015, S. 1026–1034.

[He+16] K. He, X. Zhang, S. Ren und J. Sun. „Deep residual learning for image recognition“. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, S. 770–778.

Literatur III

[Ily+18] A. Ilyas, L. Engstrom, A. Athalye und J. Lin. „Black-box adversarial attacks with limited queries and information“. In: *International conference on machine learning*. PMLR. 2018, S. 2137–2146.

[KGB+16] A. Kurakin, I. Goodfellow, S. Bengio u. a. *Adversarial examples in the physical world*. 2016.

[KMP20] V. Krishnan, A. A. A. Makdah und F. Pasqualetti. „Lipschitz Bounds and Provably Robust Training by Laplacian Smoothing“. In: *arXiv preprint arXiv:2006.03712* (2020).

[Mad+17] A. Madry, A. Makelov, L. Schmidt, D. Tsipras und A. Vladu. „Towards deep learning models resistant to adversarial attacks“. In: *arXiv preprint arXiv:1706.06083* (2017).

Literatur IV

[RKH19] K. Roth, Y. Kilcher und T. Hofmann. „Adversarial Training is a Form of Data-dependent Operator Norm Regularization“. In: *NeurIPS*. 2019.

[SGS15] R. K. Srivastava, K. Greff und J. Schmidhuber. „Highway networks“. In: *arXiv preprint arXiv:1505.00387* (2015).

[SV18] K. Scaman und A. Virmaux. „Lipschitz regularity of deep neural networks: analysis and efficient estimation“. In: *arXiv preprint arXiv:1805.10965* (2018).

[Sze+13] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow und R. Fergus. „Intriguing properties of neural networks“. In: *arXiv preprint arXiv:1312.6199* (2013).